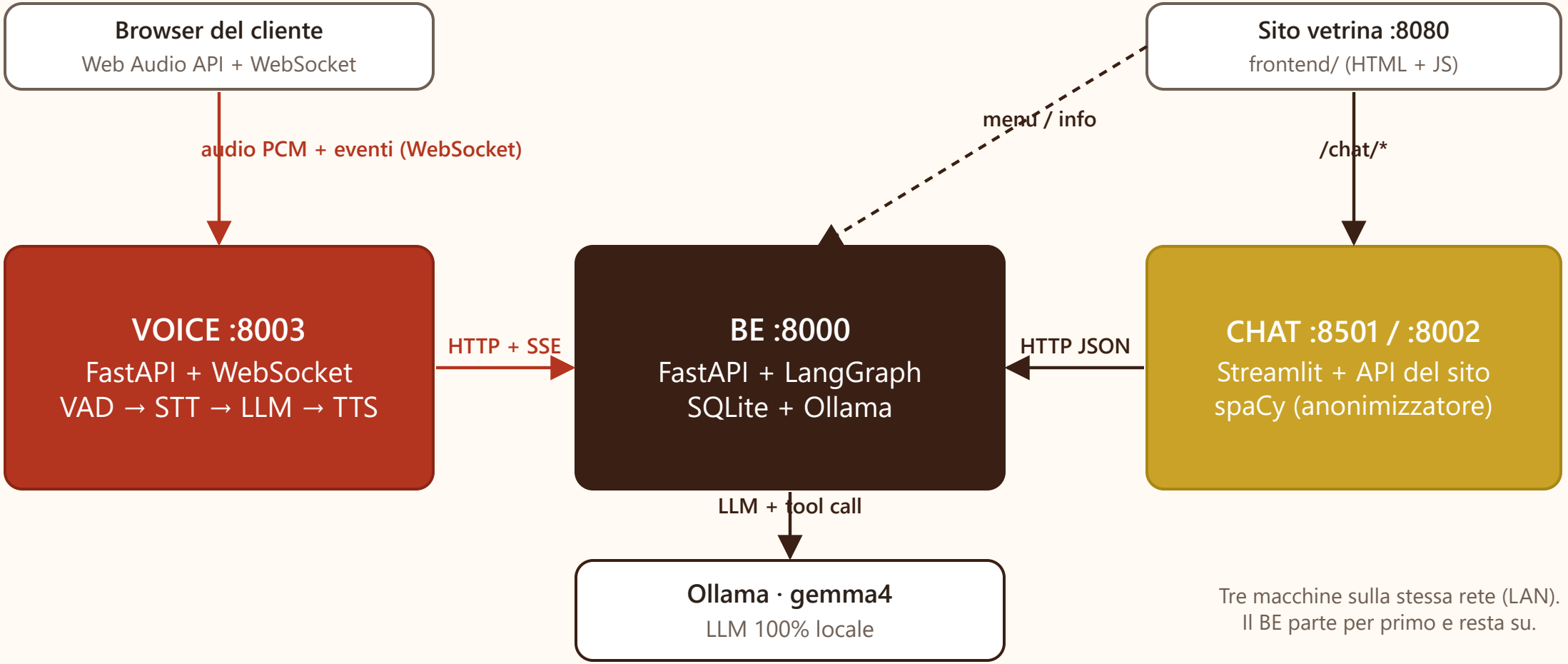
PRESENTAZIONE TECNICA

Stack tecnologico

Ambiente (venv), rete (LAN) e componente Voice: cosa usiamo, come funziona e perché l'abbiamo scelto

Il sistema in una pagina



Tre macchine sulla stessa rete (LAN).
Il BE parte per primo e resta su.

Ambiente: un venv condiviso, requisiti per ruolo

Un solo `.venv` alla radice del repo

- **base.txt** (sempre): fastapi, uvicorn[standard], httpx, pydantic, pydantic-settings, python-dotenv
- **be.txt**: langchain, langgraph, langgraph-checkpoint-sqlite, langchain-ollama, sqlalchemy
- **chat.txt**: streamlit, requests, spacy, phonenumbers
- **voice.txt**: websockets, numpy, faster-whisper, piper-tts, onnxruntime, sounddevice, phonenumbers, webrtcvad-wheels *(opzionale)*
- **dev.txt**: pytest, pytest-asyncio, mypy

Come resta coerente

- Ogni macchina installa **base + il suo ruolo** (venv leggero)
- `.venv/.setup_state.json` salva lo **sha256** di ogni file requirements
- Se un collega cambia un file, al prossimo avvio `is_setup()` vede l'hash diverso e riparte da solo
- `--role all` = tutto, per sviluppo e test
- Pacchetti che richiedono compilatore C++ sono **opzionali**: se falliscono si va avanti con un avviso

Python ≥ 3.10 (consigliato 3.11/3.12; qui 3.13.5). Il setup usa **solo la libreria standard**: funziona anche con un venv rotto.

setup/setup_env.py: cosa fa, in ordine

1 Controllo Python
≥ 3.10

2 Crea .venv
se manca

3 pip install
pacchetto per pacchetto, output live

4 Editable install
pip install -e voice

5 Modelli
spaCy · Piper · Ollama

6 Scrive lo stato
hash dei requirements

Se un pacchetto richiesto fallisce
il setup si ferma con il nome del pacchetto e riparte da lì: i passi già fatti si saltano

Se un pacchetto OPZIONALE fallisce
es. webrtcvad senza compilatore C++ → avviso e si continua: il VAD a energia è il default

Download dei modelli: voce Piper italiana da Hugging Face (~28 MB) in voice/models/ · modello spaCy it_core_news_md (solo CHAT) · Ollama: ollama pull del modello (solo BE)

entrypoint/avvia.py: un comando per macchina

1 git pull --ff-only

2 Identità: ruolo + IP

3 Setup venv (per ruolo)

4 .env di macchina

5 Firewall (netsh)

6 Attesa del BE

7 Avvio componente

git: solo fast-forward
mai merge automatici sporchi

lan/macchina.json
ruolo + IP rilevato (socket UDP)

.env generato da .env.example
BACKEND_BASE_URL=http://IP-BE:8000

Firewall
regola TCP in ingresso per ogni porta

Cancello sul BE
polling di /api/health ogni 2 s

Comandi

python entrypoint\avvia.py
--status stato delle 3 macchine
--attendi il presentatore aspetta
--sono identifica e basta
--no-git --no-wait --yes

Avvio automatico
Scheduled Task al login +30 s:
la macchina parte da sola

L'entrypoint nel dettaglio: funzione per funzione

#	Funzione	Cosa fa	Se qualcosa va storto
1	`git_pull()`	`git pull --ff-only`	si ferma e stampa l'errore: **non forzare**
2	`resolve_role()` · `ensure_identity()`	ruolo da `--role` / `LAN_ROLE` / `macchina.json` / domanda; IP rilevato	lo chiede una sola volta
3	`ensure_environment(role)`	confronta gli hash e installa solo se serve	si ferma: rilanciare dopo aver sistemato
4	`configure_env()`	scrive `BACKEND_BASE_URL`, `BE_URL`, `HOST` nel `.env`	errore solo se manca `.env.example`
5	`firewall_hint()`	`netsh advfirewall` per ogni porta del ruolo	**non blocca**: stampa il comando da admin
6	`wait_for_be()`	`GET /api/health` ogni 2 s, max 180 s (solo CHAT e VOICE)	«BE assente»: il componente non parte
7	`start_component()`	processo principale + servizi extra, log live	`Ctrl+C` ferma tutto

Codice: `entrypoint/avvia.py` (`main`) e `lan/machine_setup.py`. `--status` e `--attendi` usano gli IP di `lan_config.json`: con IP statici si fissano con `--pubblica-ip`.

LAN: ruoli, porte e cancello sul BE

Ruolo	Cartella	Porta	Health	Servizi aggiuntivi
BE	`backend/`	8000	`/api/health`	+ sito vetrina :8080 (http.server su frontend/)
CHAT	`chat/`	8501 (Streamlit)	`/_stcore/health`	+ API del sito :8002 (uvicorn chat/api.py)
VOICE	`voice/`	8003	`/health`	WebSocket /ws/voice · sito / · test /test-voice

Regole di avvio

- Ordine: **BE → CHAT → VOICE** (`ordine_avvio` in lan_config.json)
- CHAT e VOICE **non partono** finché il BE non risponde (gate di 180 s)
- Tutto è dichiarato in `lan/lan_config.json`: comando, cwd, porte, health, variabili

Perché così

- Gli IP in lan_config.json sono **esempi**: quelli veri si chiedono una volta e si ricordano in macchina.json (non in git)
- Così il giorno della demo il presentatore tocca **un solo PC**: `avvia.py --attendi`

Voice: lo stack a strati

Browser / client	Web Audio API (ScriptProcessor, 2048 campioni) · WebSocket · canvas (orb) live_client: microfono e altoparlante con sounddevice
Trasporto	FastAPI 0.143 + uvicorn 0.54 · websockets 16 asyncio.Queue con dimensione massima (backpressure)
Audio locale	VAD a energia RMS (opzionale webrtcvad) · faster-whisper 1.2.1 (CTranslate2): small, int8, CPU Piper 1.8 + onnxruntime 1.31 · voce italiana it_IT-riccardo-x_low
Conversazione	orchestrator · contacts (phonenumbers) · anonymizer (regex + spaCy 3.8) toolguard · prompt in italiano · messages / spoken (numeri e date a parole)
Client del BE	httpx 0.28 asincrono · retry con backoff · SSE (/api/chat/stream) pydantic-settings per la configurazione (.env)
Qualità	pytest 9 + pytest-asyncio · mypy strict (29 file) · log JSON strutturati doctor / diag · Dockerfile multi-stage, utente non-root, healthcheck

Qualità: test, tipi e diagnosi

192
test automatici (voice)

29
file con mypy --strict

12
controlli del doctor

3
ruoli: be · chat · voice

Come proviamo senza il resto del sistema

- **Stub BE** (voicebot/devtools/stub_be.py): un BE finto con registrazione, conferma sì/no, modifica, cancellazione
- Test **ermetici**: conftest isola dal .env e spiega l'interprete sbagliato
- `voice local`: stub + server + browser con un comando

Come capiamo cosa non va

- `python -m voicebot doctor`: python, .env, pacchetti, modelli, audio, porta, BE, registrazione
- `/diag` e tab STATO della pagina di test mostrano lo stesso report
- Log JSON a una riga: facili da filtrare

Decisioni di stack e motivi

Scelta	Alternativa scartata	Perché
Tutto locale (Ollama, Whisper, Piper)	API cloud	Vincolo di progetto: nessun dato personale fuori dalla rete
Un venv, requisiti per ruolo + hash	Un requirements.txt unico	Evita conflitti di merge e installazioni pesanti inutili
Setup e LAN in sola stdlib	Script con dipendenze	Funzionano anche quando il venv è rotto
asyncio con code limitate	Thread / code illimitate	Barge-in e backpressure semplici, niente memoria che cresce
VAD a energia di default	webrtcvad obbligatorio	Zero dipendenze compilate su Windows
Whisper small (non tiny)	tiny (più veloce)	tiny sbaglia numeri e nomi: per prenotare non basta
Pacchetti opzionali tollerati	Setup che si blocca	Un compilatore C++ mancante non deve fermare la demo

Comandi da ricordare

Cosa voglio fare	Comando
Prima volta su una macchina	`python entrypoint\avvia.py`
Solo il setup, per ruolo	`python setup\setup_env.py --role voice --with-dev`
Verifica veloce (esce 0/1)	`python setup\setup_env.py --check-only`
Test completo in locale (stub BE)	`voice\voice local`
Voice contro il BE vero	`voice\voice real → http://localhost:8003/test-voice`
Perché non funziona?	`python -m voicebot doctor`
Test + tipi	`cd voice · python -m pytest tests -q · python -m mypy src`
Stato delle tre macchine	`python entrypoint\avvia.py --status`